

Classes and Objects

Recap of Spot (Processing)

Produced Dr. Siobhán Drohan
by: Ms. Mairead Meagher

Topics list

1. Recap: **Classes and Objects**

2. Recap on the **Spot** class:

- v1.0 (**default constructor**)
- v2.0 (**constructor with parameters**)
- v3.0 (**overloading constructors**)

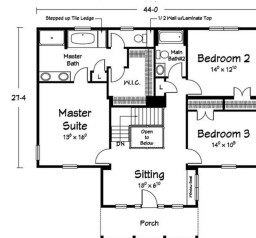
Building Analogy

A **class** is like a **blueprint** for a building.



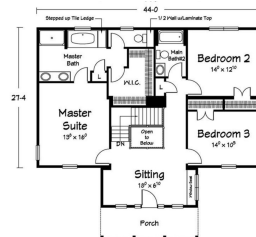
Building Analogy

An **object** is a **building** constructed from that blueprint.



Building Analogy

You can build lots of
(buildings) **objects**
from a single blueprint.



Cake Analogy

A **class** is like a
recipe for a
cake.

What you need:

- 175g/6oz Odlums Cream Plain Flour
- 75g/3oz Plain Chocolate (min 70% cocoa)
- 200g/7oz Butter
- 175g/6oz Shamrock Golden Caster Sugar
- 3 Large Eggs
- 1 teaspoon Baking Powder
- 100g packet Shamrock Ground Almonds
- 2 tablespoons Cocoa, sieved
- 2 tablespoons Milk
- 1 teaspoon Goodall's Vanilla Essence

For Chocolate Cream

- 140ml Cream
- 175g/6oz Plain Chocolate (min 70% cocoa)

How to:

1. Preheat oven to 190°C/375°F/Gas 5. Lightly grease and base line a 23cm/9" deep sandwich tin.
2. Break the chocolate into a heatproof bowl. Add 25g/1oz of the butter and stand bowl over a pan of hot water until chocolate has melted.
3. Meanwhile put the remaining butter, sugar, eggs, flour, baking powder, ground almonds, cocoa, milk and essence into a large bowl and beat until smooth and creamy.
4. Add the melted chocolate and gently stir into the mixture. Transfer to the prepared tin and level the top.
5. Bake for about 40 minutes until risen and the surface feels firm to the touch. Remove from oven. Allow to sit in tin for about 5 minutes, then transfer to a wire tray to cool.
6. Make the chocolate cream by heating the cream until just bubbling around the edges. Add the chocolate and gently stir over a low heat until melted. Remove from heat.
7. Transfer to a bowl to allow to cool and begin to set.
8. Slice cake horizontally and use half the icing to sandwich the cake.
9. Spread remaining icing on top and sides of cake.
10. Serve with raspberries and crème fraîche or Greek yoghurt.

Cake Analogy

An **object** is the
cake baked from
that recipe.



Cake Analogy

You can bake **lots of**
(cakes) objects
from a single recipe.



Cake Analogy

Each (cake) **object**
has its own
object state
(each cake looks
different here!).



Class names:

- should match its purpose.
- can be any word or combination of words e.g.:
 - Account
 - CurrentAccount
- Should begin with a **Capital letter** and not be pluralised e.g.:
 - **S**pot
 - **P**roduct

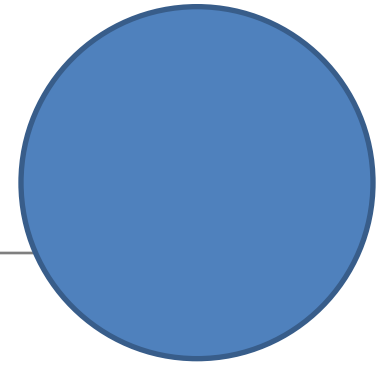
Spot – classes and objects

A **class** defines a group of related **methods** (functions) and **fields** (variables / properties).

```
class Spot
{
    float xCoord;
    float yCoord;
    float diameter;

    Spot()
    {
    }
}
```

Spot – classes and objects

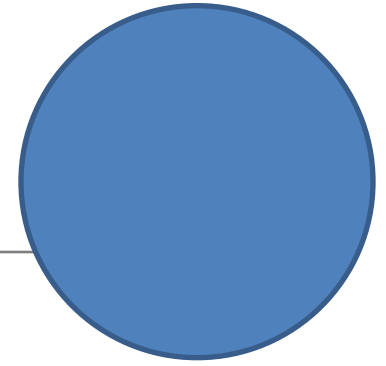


Class Name	Spot
Fields (variables, properties)	xCoord yCoord diameter
Methods (functions)	Spot()

```
class Spot
{
    float xCoord;
    float yCoord;
    float diameter;

    Spot()
    {
    }
}
```

Spot – classes and objects



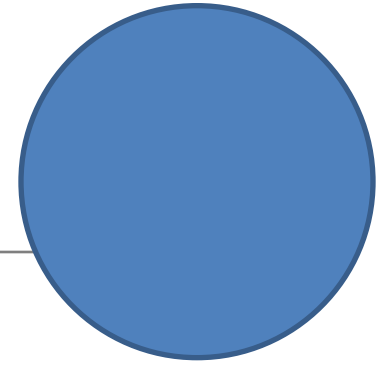
Class Name	Spot
Fields (variables, properties)	xCoord yCoord diameter
Methods (functions)	Spot()

Class Type

```
class Spot
{
    float xCoord;
    float yCoord;
    float diameter;

    Spot()
    {
    }
}
```

Spot – classes and objects



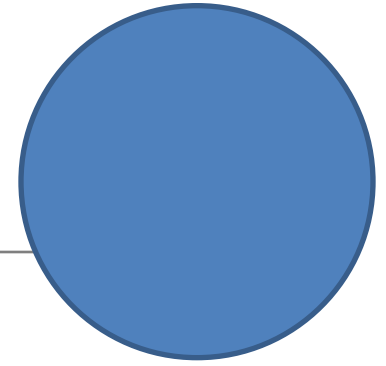
Class Name	Spot
Fields (variables, properties)	xCoord yCoord diameter
Methods (functions)	Spot()

Properties /
Attributes

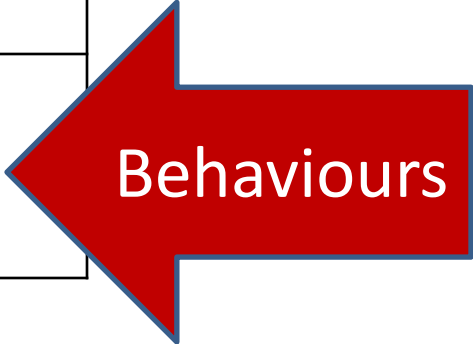
```
class Spot
{
    float xCoord;
    float yCoord;
    float diameter;

    Spot()
    {
    }
}
```

Spot – classes and objects



Class Name	Spot
Fields (variables, properties)	xCoord yCoord diameter
Methods (functions)	Spot()



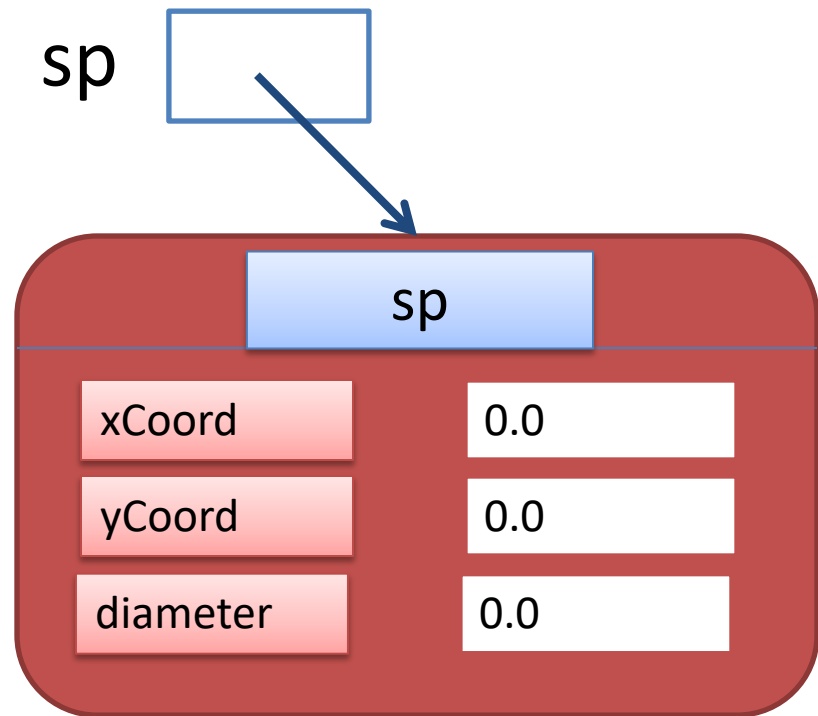
Behaviours

```
class Spot
{
    float xCoord;
    float yCoord;
    float diameter;

    Spot()
    {
    }
}
```

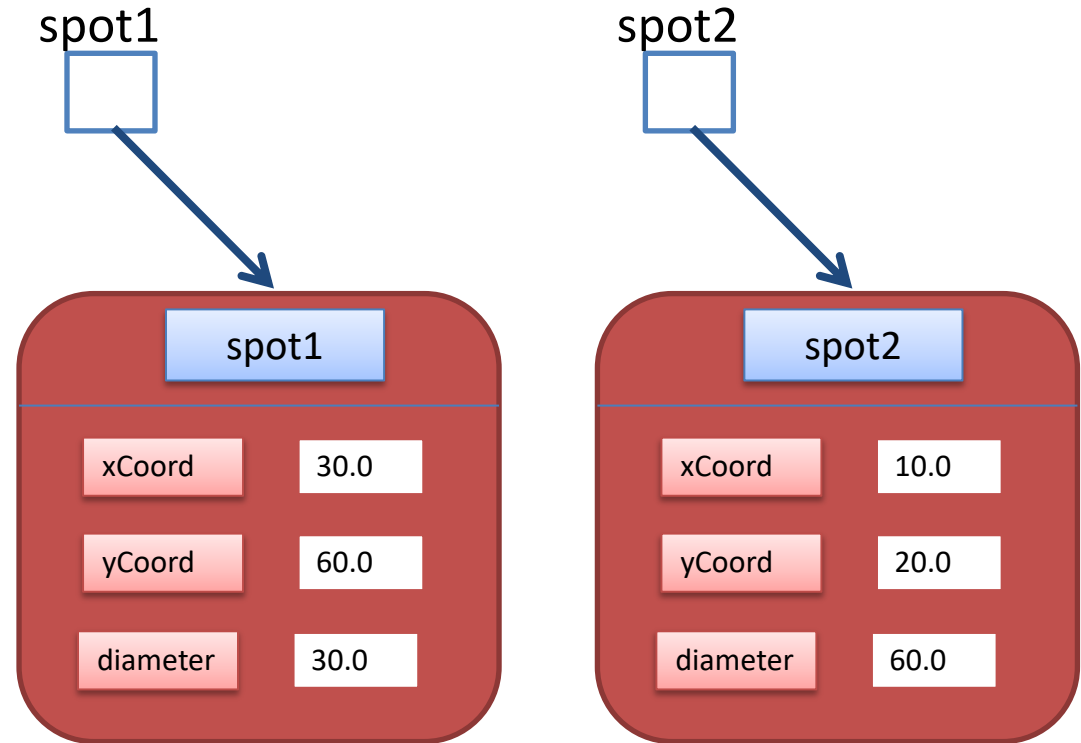
Spot – classes and objects

- An **object**
 - is a single instance of a class
 - i.e. an object is created (instantiated) from a class.



Spot – classes and objects

- Many **objects** can be constructed from a single **class** definition.

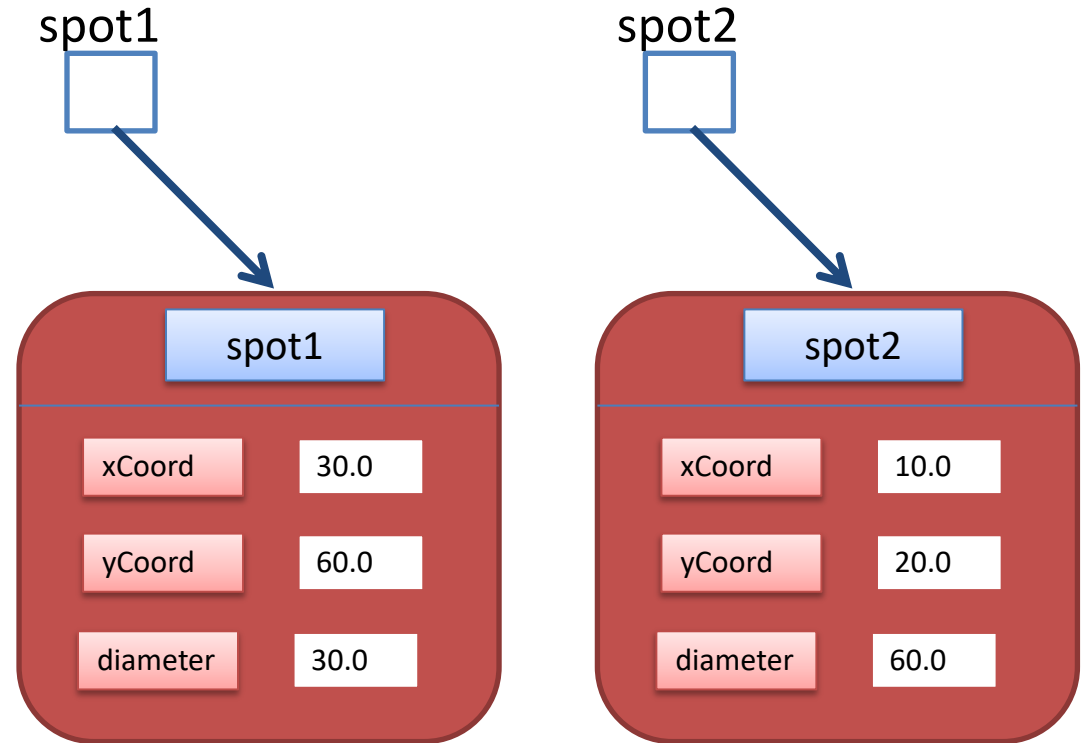


Spot – classes and objects

- Many **objects** can be constructed from a single **class** definition.

Two objects.

Each has a unique name, *spot1* and *spot2*.
Each has its own copy (values) of the fields.



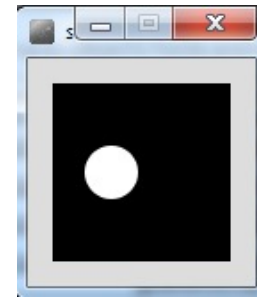
Topics list

1. Recap: **Classes and Objects**

2. Recap on the **Spot** class:

- v1.0 (**default constructor**)
- v2.0 (**constructor with parameters**)
- v3.0 (**overloading constructors**)

Spot Class – Version 1.0

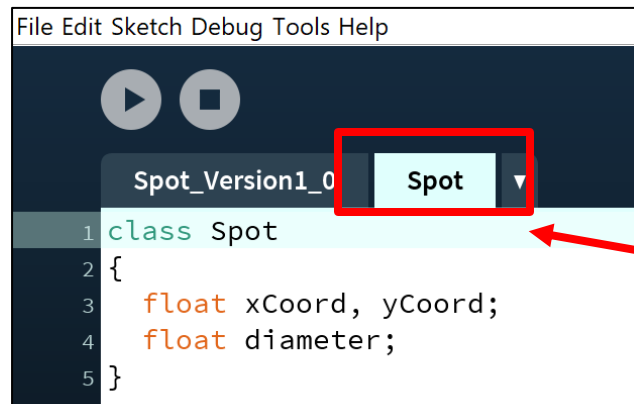


Defining the **class**

```
class Spot
```

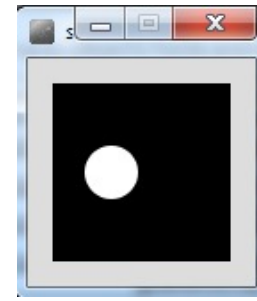
```
{  
  float xCoord, yCoord;  
  float diameter;  
}
```

Declaring the **fields** in the class



In the PDE, place this code in a new **tab**, called Spot

Spot Class – Version 1.0



Declaring an object **sp**,
of type **Spot**.

Spot sp;

Calling the **Spot()**
constructor to build the
sp object in memory.

sp = new Spot();

Initialising the fields in
the **sp** object with a
starting value.

sp.xCoord = 33;
sp.yCoord = 50;
sp.diameter = 30;

Calling the ellipse
method, using the fields
in the **sp** object as
arguments.

ellipse(sp.xCoord, sp.yCoord,
sp.diameter, sp.diameter);

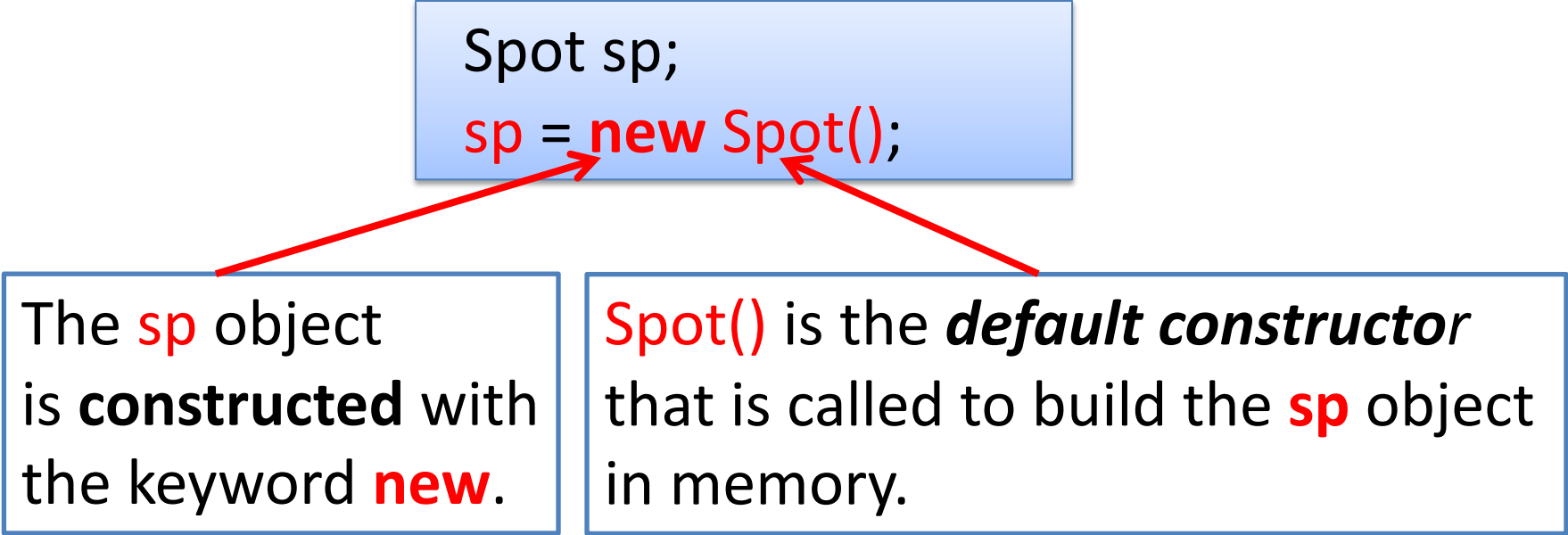
```
class Spot
{
  float xCoord, yCoord;
  float diameter;
}
```

```
void setup(){
  size (100,100);
  noStroke();
}
```

```
void draw(){
  background(0);
}
```

Constructors

```
Spot sp;  
sp = new Spot();
```



The **sp** object is **constructed** with the keyword **new**.

Spot() is the *default constructor* that is called to build the **sp** object in memory.

Constructors

```
Spot sp;  
sp = new Spot();
```

The diagram illustrates the components of object construction in Java. A top box contains the code 'Spot sp;' and 'sp = new Spot();'. Two arrows originate from this box: one points to a box explaining the 'new' keyword and the 'sp' object, and another points to a box explaining the 'Spot()' method as the default constructor. A third arrow points from the 'Spot()' box to a final box defining what a constructor is and showing the constructor's syntax.

The **sp** object is **constructed** with the keyword **new**.

Spot() is the *default constructor* that is called to build the **sp** object in memory.

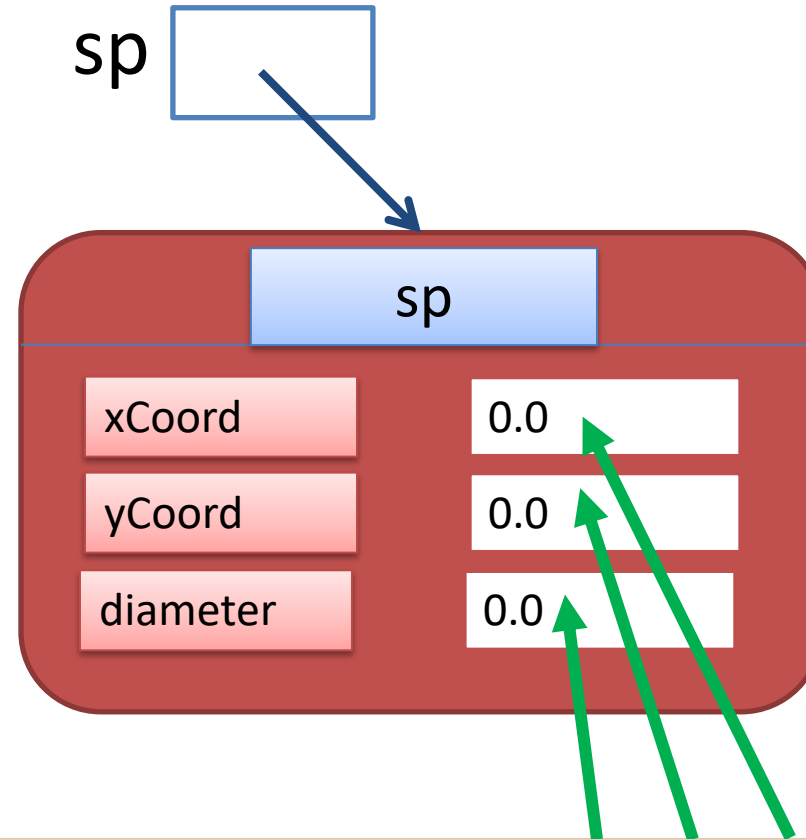
A CONSTRUCTOR is a method that has the **same name as the class** but has **no return type**.

```
Spot()  
{  
}
```

Default Constructor

```
class Spot
{
    float xCoord;
    float yCoord;
    float diameter;

    //Default Constructor
    Spot()
    {
    }
}
```



- The constructor stores initial values in the fields.

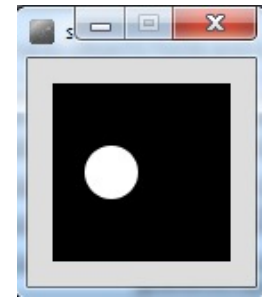
Topics list

1. Recap: **Classes and Objects**

2. Recap on the **Spot** class:

- v1.0 (**default constructor**)
- v2.0 (**constructor with parameters**)
- v3.0 (**overloading constructors**)

Spot Class – Version 2.0



```
Spot sp;

void setup()
{
  size (100,100);
  noStroke();
  sp = new Spot (33, 50, 30);
}

void draw()
{
  background(0);
  ellipse(sp.xCoord, sp.yCoord, sp.diameter, sp.diameter);
}
```

```
class Spot
{
  float xCoord, yCoord;
  float diameter;

  Spot (float xPos, float yPos, float diamtr)
  {
    xCoord = xPos;
    yCoord = yPos;
    diameter = diamtr;
  }
}
```

Topics list

1. Recap: **Classes and Objects**

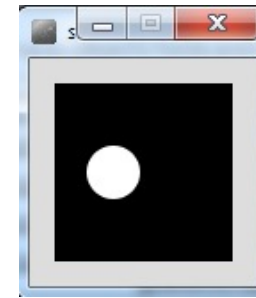
2. Recap on the **Spot** class:

- v1.0 (**default constructor**)
- v2.0 (**constructor with parameters**)
- v3.0 (**overloading constructors**)

Overloading Constructors

- We can have as many constructors as our design requires, ONCE they have unique parameter lists.
- We are **overloading** our constructors in Version 3.0...

Spot Class – Version 3.0



overloading

```
class Spot{  
    float xCoord, yCoord;  
    float diameter;
```

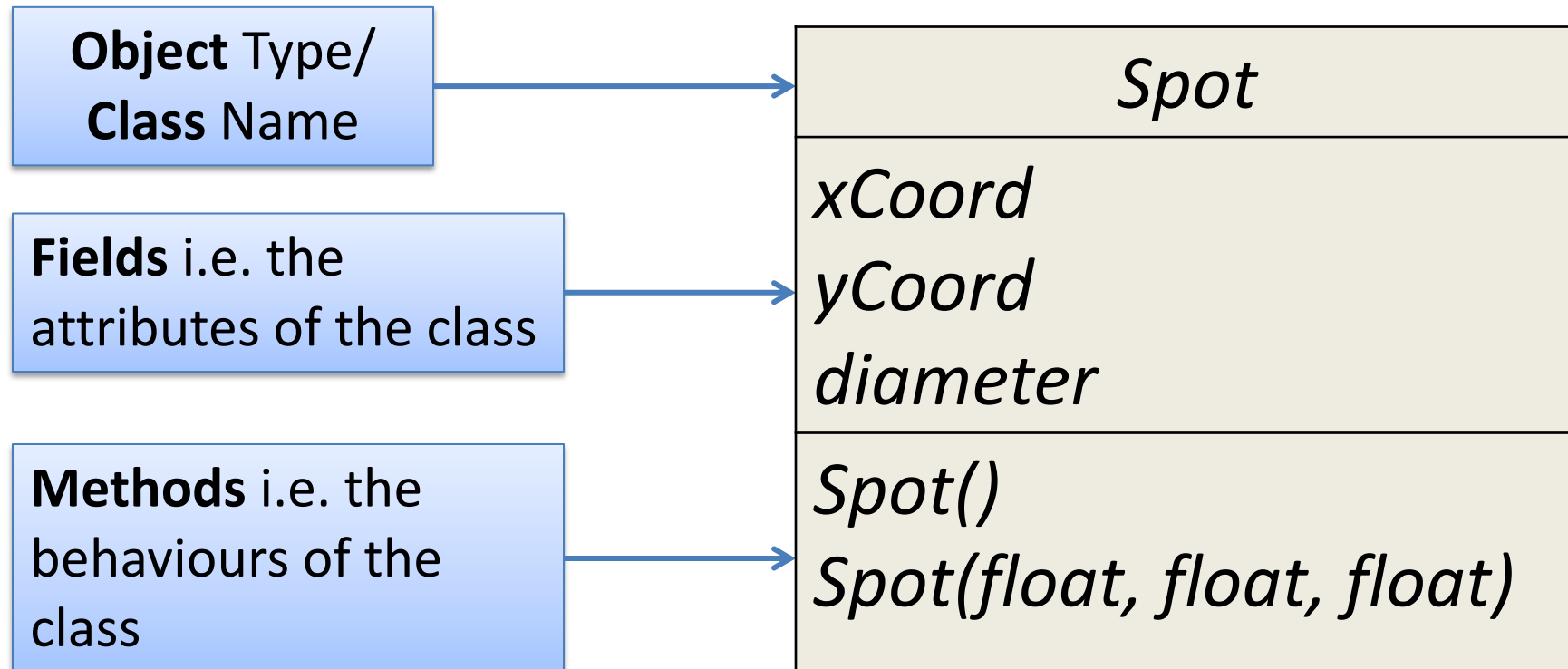
```
    Spot() {  
    }  
}
```

Default Constructor with
NO parameters

A second Constructor with a
(float, float, float) parameter list

```
    Spot (float xPos, float yPos, float diamtr){  
        xCoord = xPos;  
        yCoord = yPos;  
        diameter = diamtr;  
    }  
}
```

Class Diagram for Spot Version 3.0



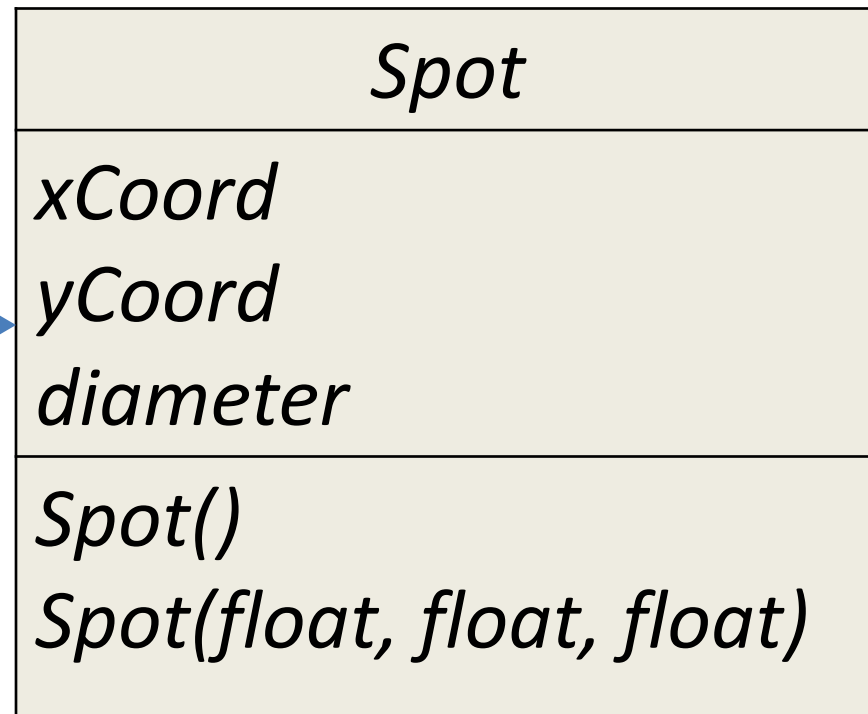
Class Diagram for Spot Version 3.0

So far, we only have overloaded constructors for our class (they create the objects of our class).

We have not defined any **behaviours** for our class e.g.

display the spot,
colour the spot,
move the spot,
and so on.

As it stands, the Spot class is not very useful!



Questions?



References

- Reas, C. & Fry, B. (2014) Processing – A Programming Handbook for Visual Designers and Artists, 2nd Edition, MIT Press, London.